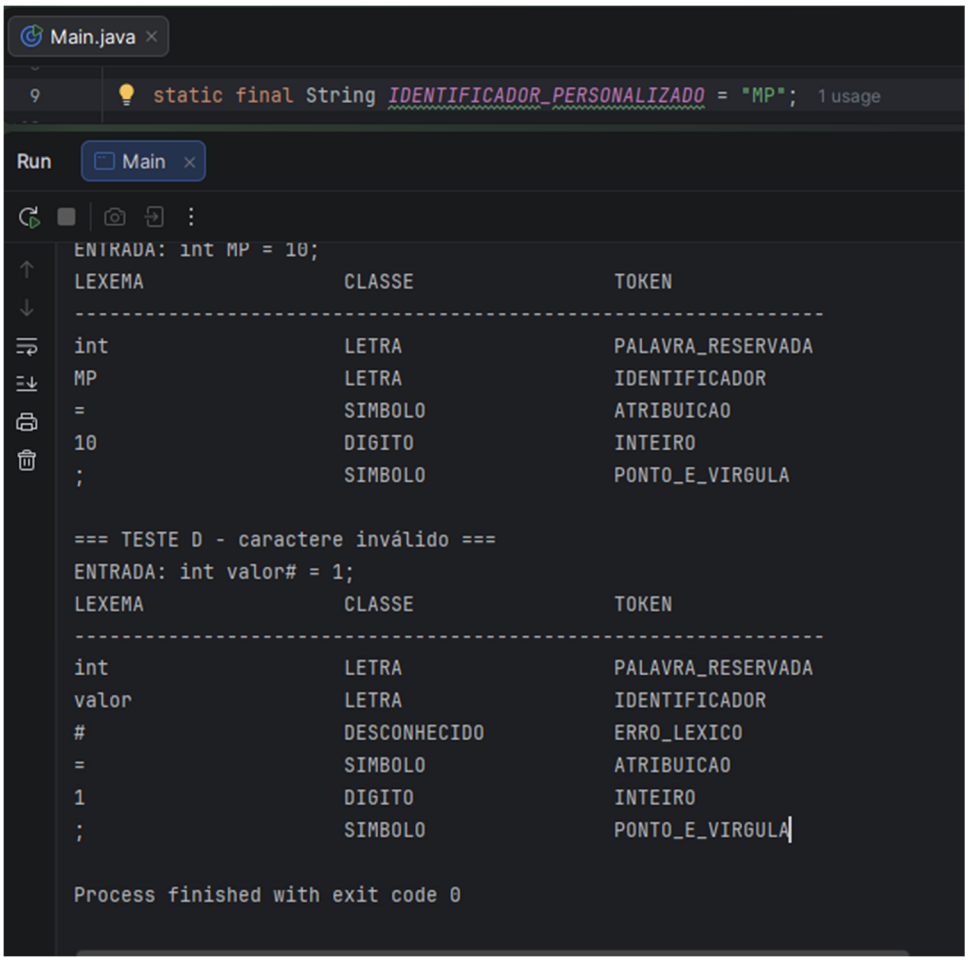
1. De caracteres a *lexemas* e *tokens*

```
ALUNO: MATHEUS_PINTOR | RA: 240005512
ENTRADA: MP = valor + 51;

LEXEMA                  TOKEN
-----------------------------------------
MP                      IDENTIFICADOR
=                       ATRIBUICAO
valor                   IDENTIFICADOR
+                       SOMA
51                      INTEIRO
;                       PONTO_E_VIRGULA

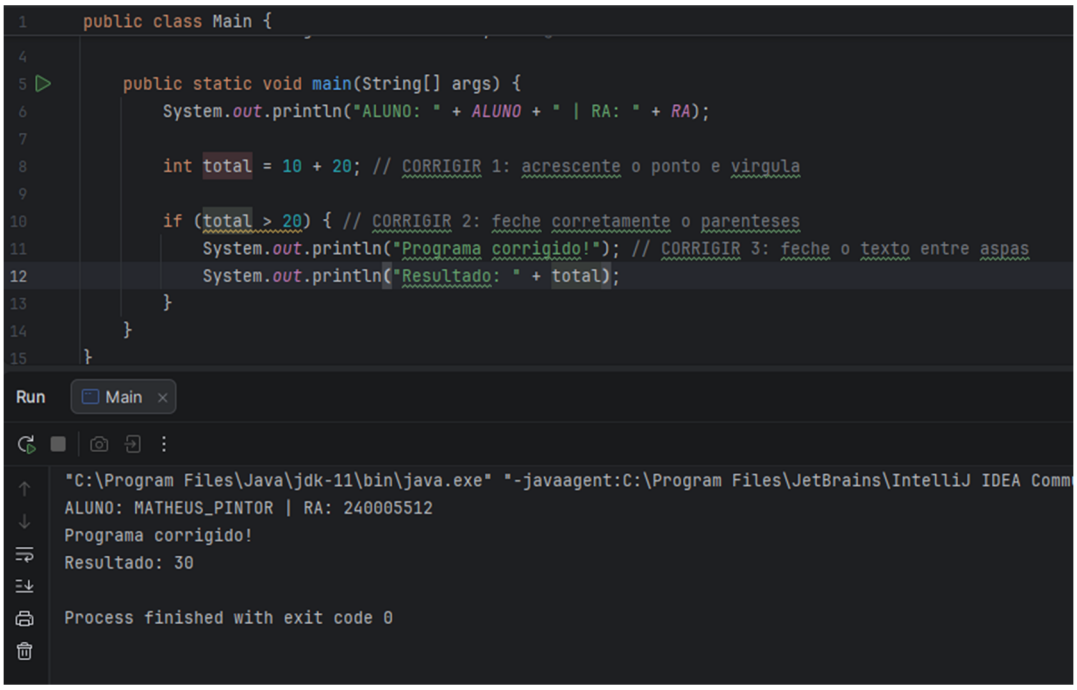
Process finished with exit code 0
```

2. O que o analisador léxico reconhece



3. Quando o compilador encontra erros





4. *Parser* descendente recursivo

Main.java

```java
public class Main {
    static final String ALUNO = "MATHEUS_PINTOR";  1 usage
    static final String RA = "240005512";  1 usage

    static final String ENTRADA = "(2 + 3) * 4";  2 usages
```

Run — Main

```
"C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Fi
ALUNO: MATHEUS_PINTOR | RA: 240005512
ENTRADA: (2 + 3) * 4

ENTRANDO em <expr> | lookahead = (
  ENTRANDO em <term> | lookahead = (
    ENTRANDO em <factor> | lookahead = (
      Abrindo parenteses
      Novo lookahead: 2
      ENTRANDO em <expr> | lookahead = 2
        ENTRANDO em <term> | lookahead = 2
          ENTRANDO em <factor> | lookahead = 2
            Numero reconhecido: 2
            Novo lookahead: +
          SAINDO de <factor> | lookahead = +
        SAINDO de <term> | lookahead = +
        Encontrado operador +
        Novo lookahead: 3
        ENTRANDO em <term> | lookahead = 3
```

5. *Parser* ascendente: deslocar e reduzir

```java
import ...

public class Main {
    static final String ALUNO = "MATHEUS_PINTOR";  1 usage
    static final String RA = "240005512";  1 usage
```

Run — Main

```
PILHA                          ENTRADA          ACAO
-------------------------------------------------------------------------------
$ 0                            id+id*id$        SHIFT id -> estado 5
$ 0 id 5                       +id*id$          REDUCE F -> id
$ 0 F 3                        +id*id$          REDUCE T -> F
$ 0 T 2                        +id*id$          REDUCE E -> T
$ 0 E 1                        +id*id$          SHIFT + -> estado 6
$ 0 E 1 + 6                    id*id$           SHIFT id -> estado 5
$ 0 E 1 + 6 id 5               *id$             REDUCE F -> id
$ 0 E 1 + 6 F 3                *id$             REDUCE T -> F
$ 0 E 1 + 6 T 9                *id$             SHIFT * -> estado 7
$ 0 E 1 + 6 T 9 * 7            id$              SHIFT id -> estado 5
$ 0 E 1 + 6 T 9 * 7 id 5       $                REDUCE F -> id
$ 0 E 1 + 6 T 9 * 7 F 10       $                REDUCE T -> T * F
$ 0 E 1 + 6 T 9                $                REDUCE E -> E + T
$ 0 E 1                        $                ACCEPT

Process finished with exit code 0
```